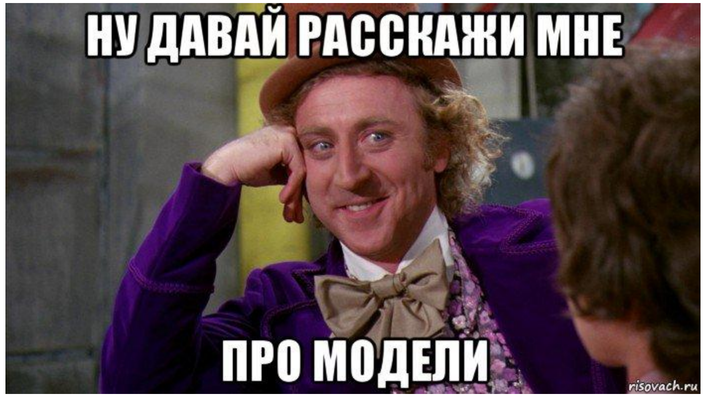
Разработка программного модуля компьютерного зрения в PyBullet.

12.02.2026

Моделирование

Моделирование – метод решения задач, при использовании которого исследуемая система заменяется более простым объектом, описывающим реальную систему и называемым моделью.

Моделирование применяется в случаях, когда проведение экспериментов над реальной системой невозможно или нецелесообразно, например из-за высокой стоимости или длительности проведения эксперимента в реальном масштабе времени.



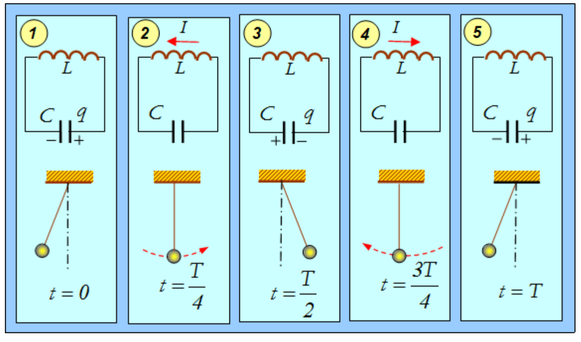
Просто мем для привлечения внимания

Моделирование

Объект и модель находятся в отношении сходства, то есть модель по каким-то признакам должна быть подобна изучаемому объекту. Это явление называют **изоморфизмом**.

подобное масштабирование

косвенное подобие



условное подобие

Программные комплексы

MATLAB
SIMULINK®

CoppeliaSim
Create. Compose. Simulate.
Any Robot.

Webots
robot simulation

MSC ADAMS

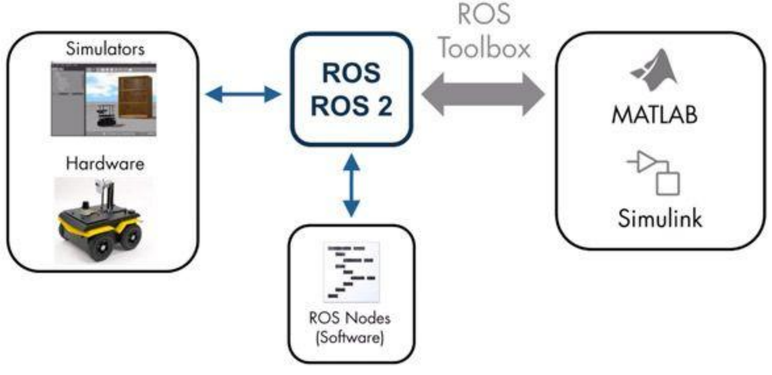
Ansys
WORKBENCH

anylogic

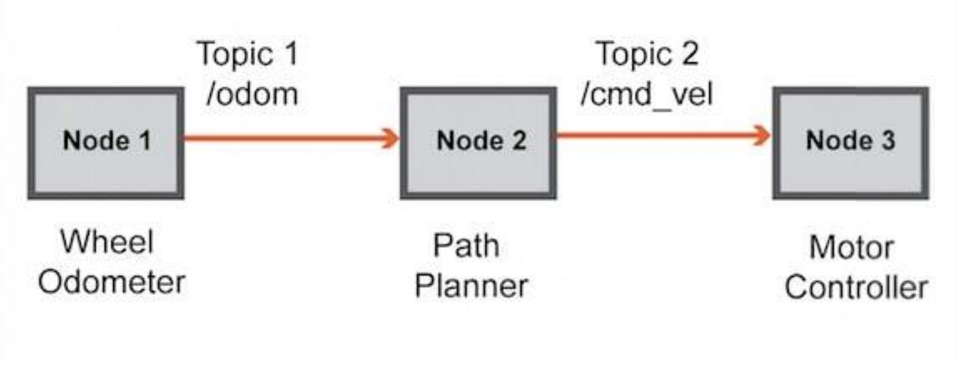
OPEN DYNAMICS ENGINE™

Программные комплексы

ROS (Robot Operating System) — это метаоперационная система с открытым исходным кодом, предназначенная для управления роботами и их программными компонентами.

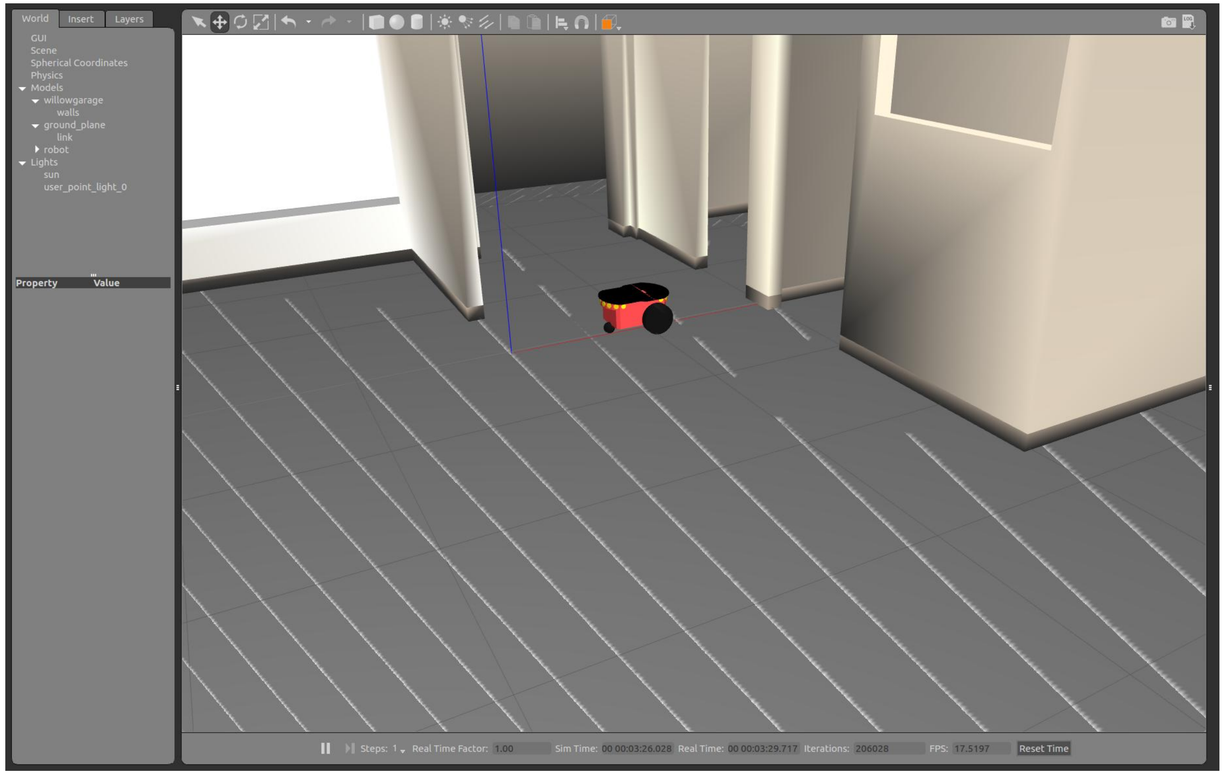


Robot Operating System – программная платформа, содержащая набор библиотек, протоколов обмена и инструментов, созданных для облегчения разработки программного обеспечения для робототехнических систем.



Программные комплексы

Gazebo - это немного большее, чем просто визуализатор; это целый симулятор мира с описанной физикой.

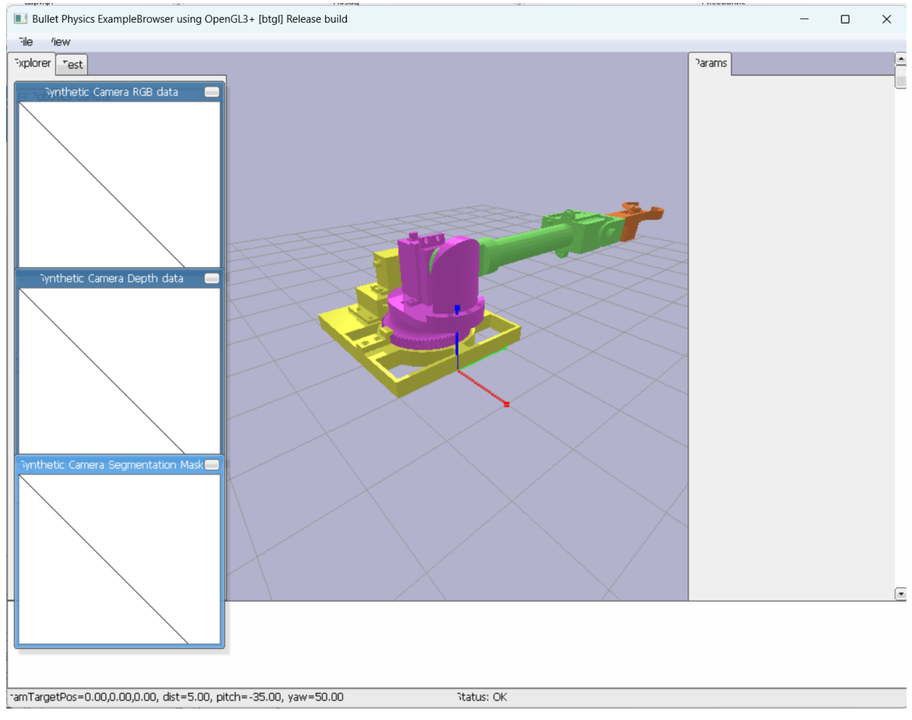


pybullet 3.2.7

```
pip install pybullet
```

Программные комплексы

```
# Инициализация PyBullet
physicsClient = p.connect(p.GUI)
p.setGravity(0, 0, -9.8)
startPos = [0, 0, 0]
startOrientation = p.getQuaternionFromEuler([0, 0, 0])
path = "manipulator/urdf/manipulator.urdf"
scale = 20
```

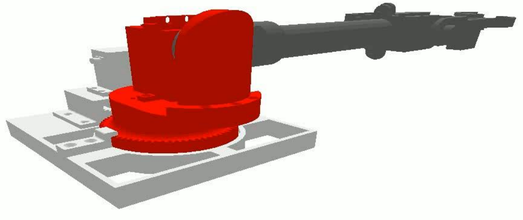


pybullet 3.2.7

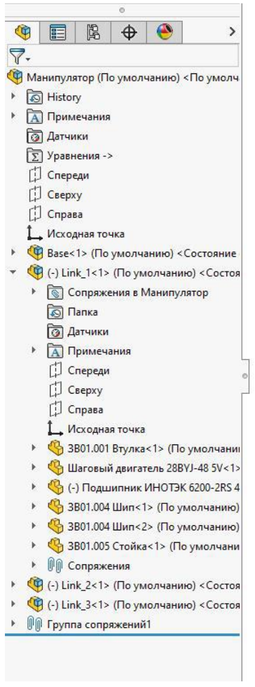
```
pip install pybullet
```

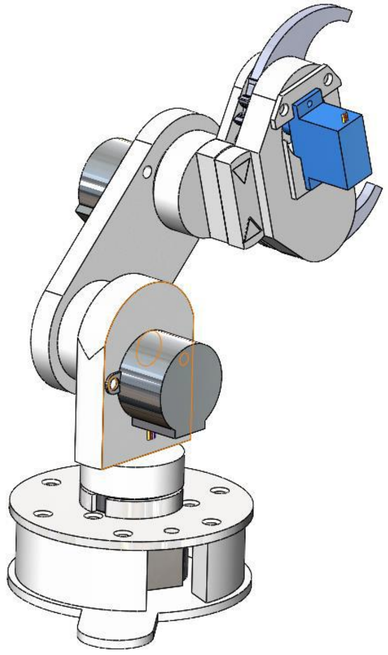
Программные комплексы

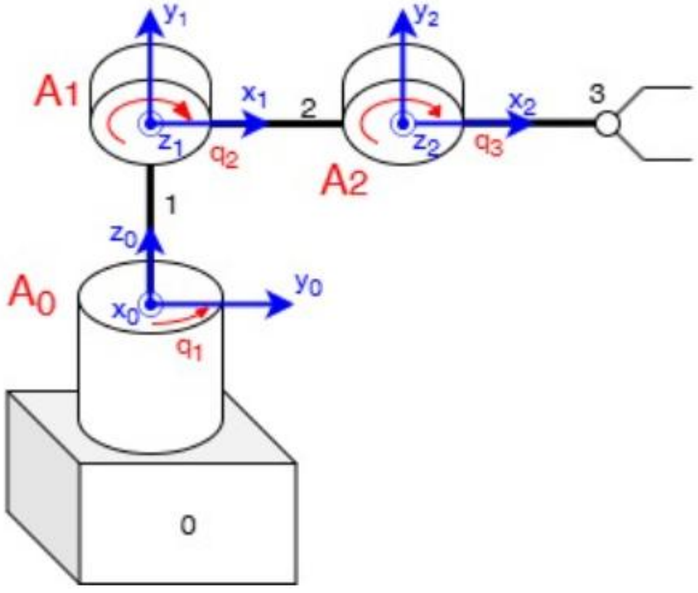
```
# Инициализация PyBullet
physicsClient = p.connect(p.GUI)
p.setGravity(0, 0, -9.8)
startPos = [0, 0, 0]
startOrientation = p.getQuaternionFromEuler([0, 0, 0])
path = "manipulator/urdf/manipulator.urdf"
scale = 20
```

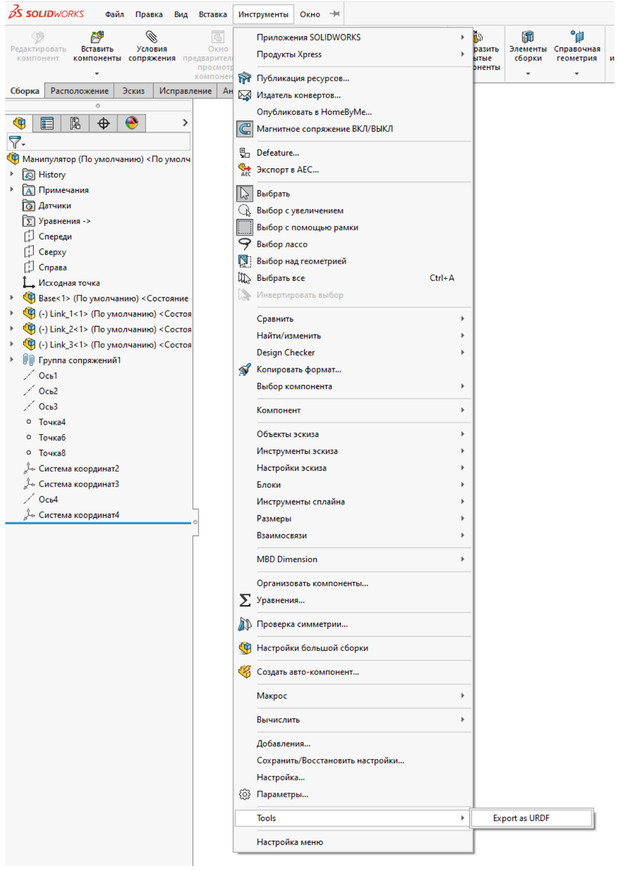


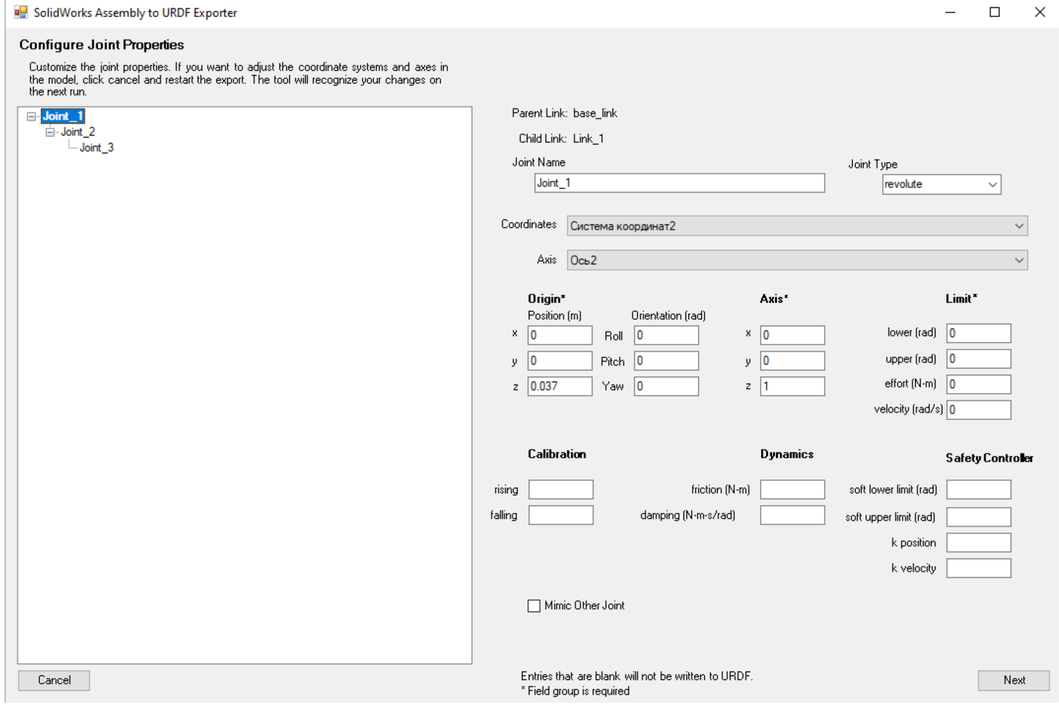
URDF


Манипулятор (По умолчанию) <По умолч
History
Примечания
Датчики
Уравнения ->
Спереди
Сверху
Справа
Исходная точка
Base<1> (По умолчанию) <Состояние
(-) Link_1<1> (По умолчанию) <Состоя
Сопряжения в Манипулятор
Папка
Датчики
Примечания
Спереди
Сверху
Справа
Исходная точка
3B01.001 Втулка<1> (По умолчани
Шаговый двигатель 28BYJ-48 5V<1>
(-) Подшипник ИНОТЭК 6200-2RS 4
3B01.004 Шип<1> (По умолчанию)
3B01.004 Шип<2> (По умолчанию)
3B01.005 Стойка<1> (По умолчани
Сопряжения
(-) Link_2<1> (По умолчанию) <Состоя
(-) Link_3<1> (По умолчанию) <Состоя
Группа сопряжений1

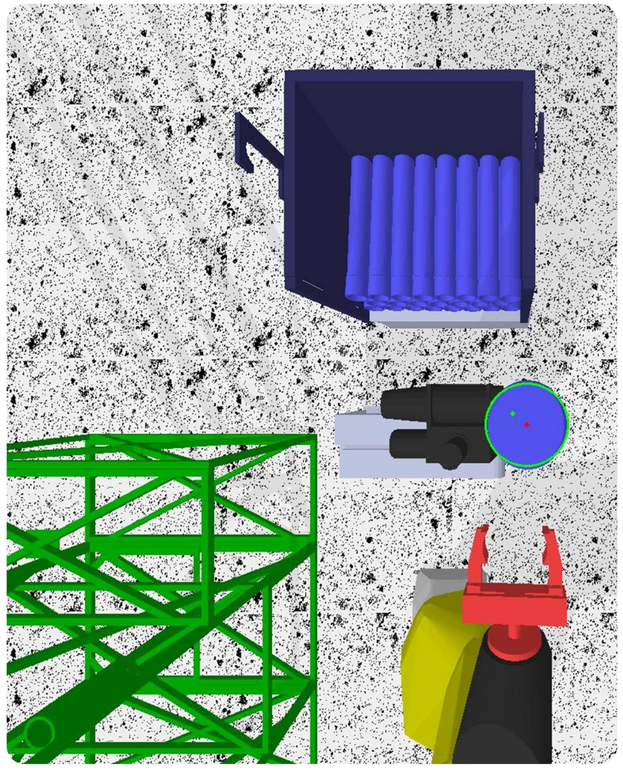



y1
y2
A1
x1
2
x2
3
z1
q2
z2
q3
A2
1
z0
A0
x0
y0
q1
0

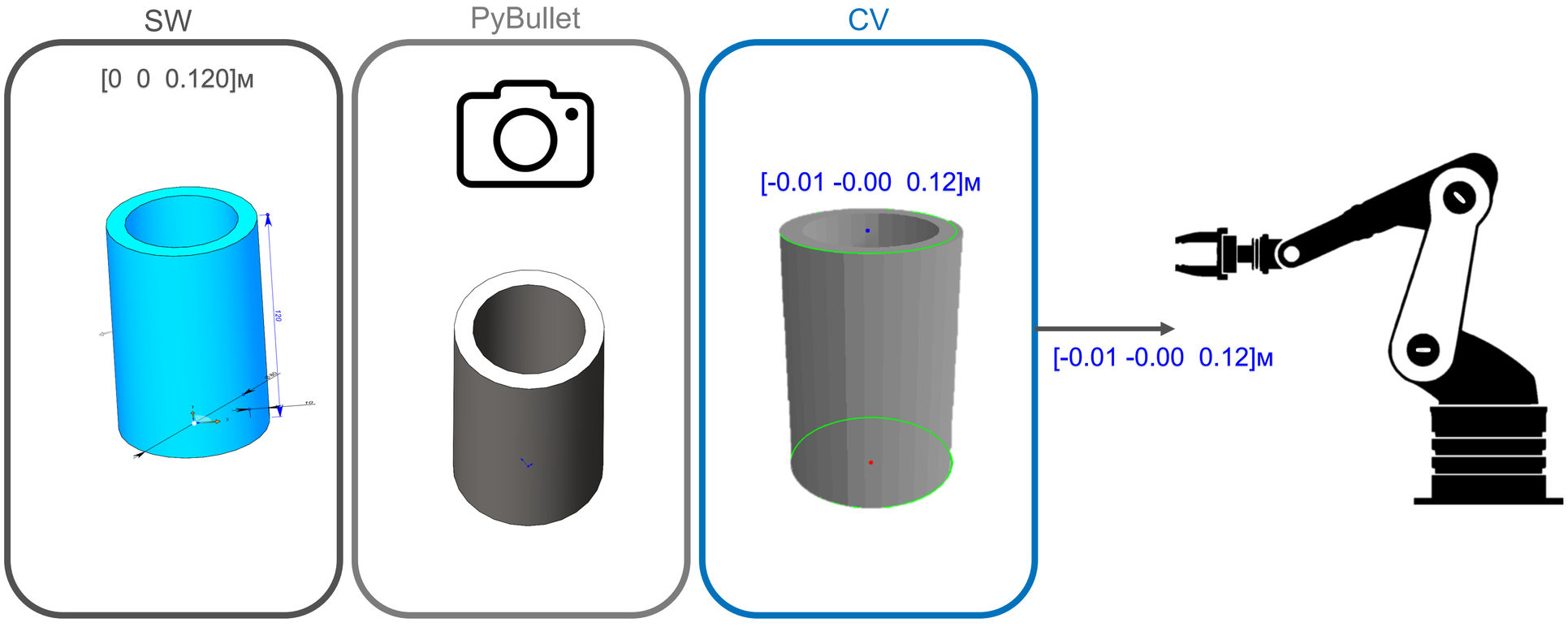

SOLIDWORKS
Файл
Правка
Вид
Вставка
Инструменты
Окно
Вставить компоненты
Условия сопряжения
Элементы сборки
Справочная геометрия
Сборка
Расположение
Эскиз
Исправление
Приложения SOLIDWORKS
Продукты Xpress
Публикация ресурсов...
Издатель конвертов...
Опубликовать в HomeByMe...
Магнитное сопряжение ВКЛ/ВЫКЛ
Defeature...
Экспорт в AEC...
Выбрать
Выбор с увеличением
Выбор с помощью рамки
Выбор лассо
Выбор над геометрией
Выбрать все
Ctrl+A
Инвертировать выбор
Сравнить
Найти/изменить
Design Checker
Копировать формат...
Выбор компонента
Компонент
Объекты эскиза
Инструменты эскиза
Настройки эскиза
Блоки
Инструменты сплайна
Размеры
Взаимосвязи
MBD Dimension
Организовать компоненты...
Уравнения...
Проверка симметрии...
Настройки большой сборки
Создать авто-компонент...
Макрос
Вычислить
Добавления...
Сохранить/Восстановить настройки...
Настройка...
Параметры...
Tools
Export as URDF
Настройка меню


SolidWorks Assembly to URDF Exporter
Configure Joint Properties
Customize the joint properties. If you want to adjust the coordinate systems and axes in the model, click cancel and restart the export. The tool will recognize your changes on the next run.
Joint_1
Joint_2
Joint_3
Parent Link: base_link
Child Link: Link_1
Joint Name
Joint_1
Joint Type
revolute
Coordinates
Система координат2
Axis
Ось2
Origin*
Position (m)
Orientation (rad)
Axis*
Limit*
x 0
y 0
z 0.037
Roll 0
Pitch 0
Yaw 0
x 0
y 0
z 1
lower (rad) 0
upper (rad) 0
effort (N-m) 0
velocity (rad/s) 0
Calibration
rising
falling
Dynamics
friction (N-m)
damping (N-m-s/rad)
Safety Controller
soft lower limit (rad)
soft upper limit (rad)
k position
k velocity
Mimic Other Joint
Cancel
Entries that are blank will not be written to URDF.
* Field group is required
Next

Постановка задачи



Постановка задачи



План действий

- Установить PyBullet
- Смоделировать втулку в SW
- Сделать URDF втулки
- Загрузить URDF файл втулки
- Задать параметры камеры в симуляции
- Сохранить изображение с камеры в симуляции
- Сохранить карту глубины из симуляции
- Обработать изображение так, чтоб найти центр верхней грани втулки
- Получить значения координат центра втулки в пикселях
- Преобразовать значения найденного центра из пикселей в метры в глобальной системе координат используя параметры камеры и знания о ее положении
- Написать отчет о проделанной работе, дополнив его рисунками с ключевыми этапами обработки изображения (кадр полученный из симуляции, кадр с найденными центрами эллипсов и тд)

План действий

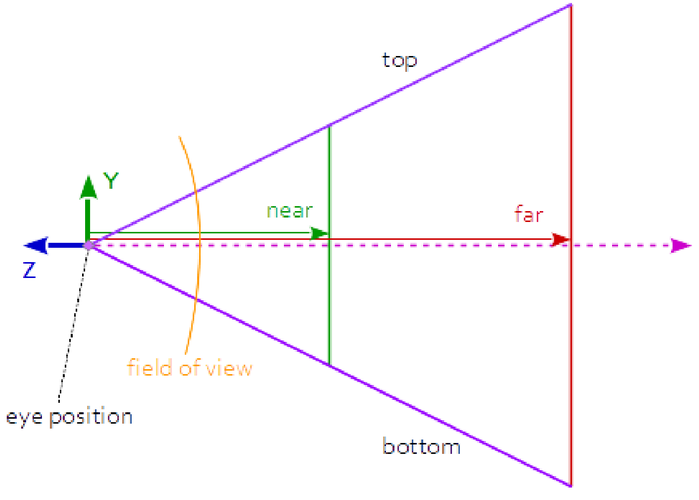
```
import pybullet as p
import time
import numpy as np
import cv2



# Инициализация PyBullet
physicsClient = p.connect(p.GUI)
p.setGravity(0, 0, -9.8)
startPos = [0, 0, 0]
startOrientation = p.getQuaternionFromEuler([0, 0, 0])
path = "C:/urdf/vtulka/urdf/Vtulka.urdf"
scale = 1.0


RoboId = p.loadURDF(path, startPos, startOrientation,
                    useFixedBase=True, globalScaling=scale)
```

План действий

computeProjectionMatrixFOV

This command also will return a 4x4 projection matrix, using different parameters. You can check out OpenGL documentation for the meaning of the parameters.
The input parameters are:

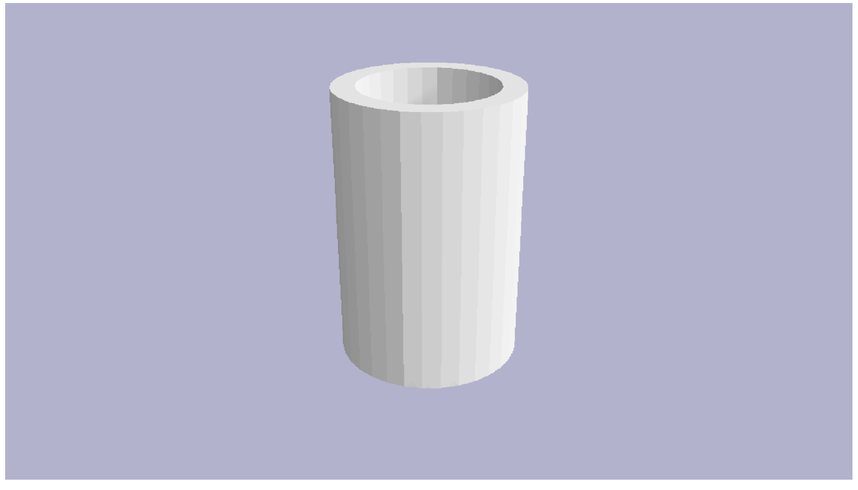
required	fov	float	field of view
required	aspect	float	aspect ratio
required	nearVal	float	near plane distance
required	farVal	float	far plane distance
optional	physicsClientId	int	unused, added for API consistency.




```
width  = 1920
height = 1080
fy = 1796.1276552044753

near = 0.001
far = 3

fov = f (height, fy)
aspect = f (width, height)


projection_matrix = p.computeProjectionMatrixFOV(fov, aspect, near, far)


eye = np.array([x.x, x.x, x.x])
target = np.array([x.x, x.x, x.x])
up_world = np.array([x.x, x.x, x.x])


view_matrix = p.computeViewMatrix(
    cameraEyePosition=eye,
    cameraTargetPosition=target,
    cameraUpVector=up_world
)
```

План действий

```
for i in range(1):
    p.stepSimulation()

    # Получаем кадр
    img = p.getCameraImage(
        width, height,
        view_matrix,
        projection_matrix,
        renderer=p.ER_BULLET_HARDWARE_OPENGL
    )
    depth_buffer = img[3]
    depth = np.reshape(depth_buffer, (height, width))

    # raw RGBA (int32)
    rgba = np.reshape(img[2], (height, width, 4)).astype(np.uint8)

    # оставляем только RGB
    rgb = rgba[:, :, :3]

    # Сохраняем
    cv2.imwrite("simulated_frame.png", cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR))

    time.sleep(1. / 240.)
```



План действий

Реконструкция координат из пикселей к метрам:

- Найдем в карте глубины (сохраняли ее в симуляции) значение соответствующие центру эллипса
- Найти функцию, связывающую центра эллипса (x, y, z, 1) пикселя с нормализованной координатой устройства (для перехода в пространство clip):

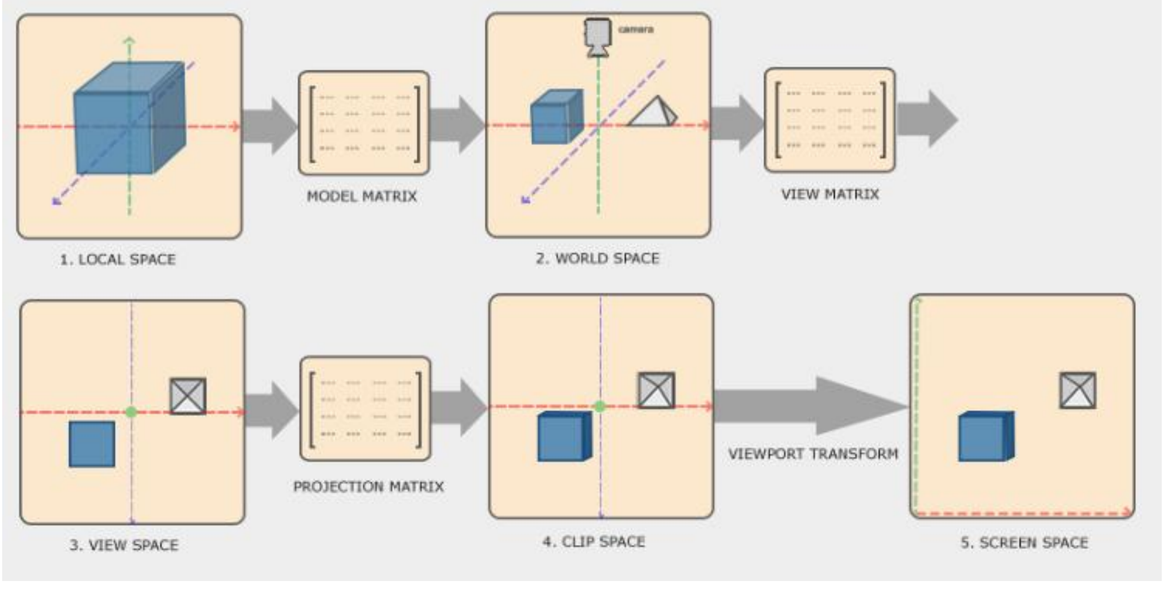
$$x \in [0, width] \rightarrow x_n \in [-1, 1]$$
$$y \in [0, height] \rightarrow y_n \in [-1, 1]$$
$$z \in [0, 1] \rightarrow z_n \in [-1, 1]$$

- Считаем обратную матрицу для projection_matrix
- Умножая на обратную матрицу к матрице проекции переходим в view space
- Выполняем перспективное деление

$$x_{vs} = \frac{x_{vs_1}}{\text{функция глубины}};$$
$$y_{vs} = \frac{y_{vs_1}}{\text{функция глубины}};$$
$$z_{vs} = \frac{z_{vs_1}}{\text{функция глубины}};$$

- Считаем обратную матрицу для view_matrix
- Умножая на обратную матрицу к view_matrix переходим в world space
- Выполняем перспективное деление
- Выводим получившиеся координаты

Полезные ссылки
https://learnopengl.com/Getting-started/Coordinate-Systems
https://www.songho.ca/opengl/gl_projectionmatrix.html



RUSSIAN SCIENCE IS FASCINATING!